SELENIUM IDE

Smart Expense Tracker

Project	Smart Expense Tracker
Document	Testing Report
Version	1.0
Date	24-04-2026
Prepared By	Agile Innovators
Subject	Agile Software Development and DevOps
Tool Used	Selenium IDE

Objective

The objective of this experiment is to perform automation testing on the Smart Expense Tracker web application using Selenium IDE. The primary goal is to validate the correctness of key functional modules — including User Registration, User Login, Dashboard Access, Budget Management, Transaction Management, and Logout — by designing structured test cases and executing them through automated scripts. This experiment also aims to demonstrate the practical distinction between manual and automation testing, survey commonly used automation testing tools, and document the step-by-step process of recording, executing, and evaluating test scenarios within Selenium IDE.

Q.1 Define Test Cases

1.1 Definition of Test Cases

A test case is a formal document that specifies a set of conditions, input values, execution steps, and expected outcomes used to verify that a particular feature or component of a software application behaves as intended. Test cases serve as the foundation of the testing process, ensuring that all requirements are systematically validated and that the system functions correctly under both normal and exceptional conditions.

1.2 Key Components of a Test Case

Each well-structured test case comprises the following essential components:

Component	Description
Test Case ID	A unique identifier assigned to each test case for reference and traceability (e.g., TC-01).
Test Title / Name	A concise description of the functionality or scenario being tested.
Module / Feature	The specific application module or feature under test (e.g., Login, Registration).
Pre-Conditions	The setup or state required before the test case can be executed.
Test Steps	A numbered, step-by-step sequence of actions to be performed during test execution.
Test Data	The specific input values used during testing (e.g., valid email, incorrect password).
Expected Result	The anticipated output or system behaviour upon execution of the test steps.
Actual Result	The observed output or system response after the test has been executed.
Status	The final result of test execution: Pass or Fail.
Priority	The importance level assigned to the test case: High, Medium, or Low.

1.3 Types of Test Cases

- Functional Test Cases — Verify that each feature performs according to specified requirements.
- UI Test Cases — Evaluate the visual appearance and usability of interface elements.
- Boundary Test Cases — Test behaviour at the boundary values of input ranges.
- Negative Test Cases — Confirm the application handles invalid inputs gracefully.
- Integration Test Cases — Validate interaction and data flow between modules.

Q.2 Differentiate Between Manual and Automation Testing

2.1 Overview

Software testing can be performed either manually by human testers or through automated scripts using dedicated testing tools. Manual testing involves a tester executing test cases step-by-step without automation tools, relying on human judgment to verify results. Automation testing utilises software tools to execute pre-written test scripts automatically, enabling faster execution, improved accuracy, and greater repeatability.

2.2 Detailed Comparison Table

Aspect	Manual Testing	Automation Testing
Execution Method	Performed manually by a human tester following documented test steps.	Executed automatically by scripts using tools such as Selenium IDE.
Speed	Relatively slow due to human execution time and cognitive processing.	Significantly faster; scripts can execute thousands of test cases in minutes.
Accuracy	Prone to human error, fatigue, and oversight, particularly in repetitive tests.	Highly accurate and consistent; produces identical results across multiple runs.
Initial Cost	Low initial cost — only requires human resources without tool investment.	Higher upfront investment in tools, infrastructure, and script development.
Reusability	Test cases must be re-executed manually each time; no inherent reusability.	Automated scripts are reusable across versions and regression cycles.
Best Suited For	Exploratory, usability, and ad-hoc testing scenarios.	Regression testing, load testing, and repetitive functional test cycles.
Maintenance	Requires manual update of test documentation for each release.	Scripts must be maintained and updated when the application changes.
Human Judgment	Highly effective for tests requiring subjective or visual evaluation.	Limited to predefined assertions; cannot assess subjective qualities.

2.3 When to Use Manual Testing

- User interface (UI) and usability testing where human judgment is essential.
- Exploratory testing when requirements are incomplete or unclear.
- Small-scale projects with limited test cycles and tight budgets.

2.4 When to Use Automation Testing

- Regression testing after each new build or release.
- Large-scale applications with an extensive suite of test cases requiring repeated execution.
- Performance and load testing to simulate large volumes of concurrent users.
- Repetitive test cases that would be tedious or error-prone if performed manually.

Q.3 Survey on Different Automation Tools Used for Software Testing

3.1 Introduction

A wide range of automation testing tools are available to support various testing requirements, including web application testing, unit testing, API testing, and performance testing. The selection of an appropriate tool depends on the technology stack, team expertise, project scale, and testing objectives.

3.2 Automation Tools Survey

Tool	Description & Key Features
Selenium IDE	A record-and-playback browser extension for web automation testing. Ideal for functional testing without programming knowledge.
Selenium WebDriver	A programmatic interface for browser automation supporting multiple languages (Java, Python, C#). Used for complex, code-driven scenarios.
TestNG	A testing framework for Java supporting test configuration, parallel execution, and detailed reporting. Commonly used with Selenium WebDriver.
JUnit	A widely adopted unit testing framework for Java, enabling developers to write and execute repeatable automated unit tests.
Cypress	A modern frontend testing framework built on JavaScript with real-time browser interaction and automatic waiting.
Playwright	A cross-browser automation library by Microsoft supporting Chromium, Firefox, and WebKit.
Appium	An open-source tool for automating mobile applications on iOS and Android platforms.
JMeter	A performance and load testing tool for web services, APIs, and database queries.

3.3 Selenium IDE Command Reference

- open — Navigates the browser to a specified URL.
- click — Simulates a mouse click on a web element identified by its locator.
- type — Enters a specified text value into an input field.
- assert — Validates a condition and halts execution if it fails.
- verify — Validates a condition but continues execution even if it fails.
- waitForElementVisible — Pauses execution until a specified element becomes visible.

3.4 Tool Selection for Smart Expense Tracker

- Ease of Use — Intuitive graphical interface; no coding required.
- Rapid Prototyping — Record-and-playback enables quick creation and execution of test cases.
- Integration — Available as a browser extension for Google Chrome and Microsoft Edge.

Q.4 Selenium IDE — Web-Based Testing on Prototype

4.1 Tools Required

Attribute	Details
Browser	Google Chrome / Microsoft Edge
Extension	Selenium IDE (Chrome Web Store)
Project Name	Smart Expense Tracker
Backend Runtime	Flask (Python)

Database	PostgreSQL (pgAdmin 4)
Frontend	HTML / CSS / JavaScript
Testing URL	http://127.0.0.1:5000

4.2 Project Description

Smart Expense Tracker is a web-based personal finance management application developed using Flask as the backend framework, PostgreSQL as the relational database, and HTML/CSS/JavaScript for the frontend interface. The application provides secure user authentication, a personalised dashboard with expense analytics, budget management, transaction tracking, and a secure logout mechanism. All testing is conducted against the locally hosted instance at http://127.0.0.1:5000.

4.3 Functional Requirements Tested

The following functional requirements were identified and subjected to automated testing using Selenium IDE:

Functional Requirement	Description
User Registration	Verifies that new users can create an account with valid details, and that appropriate errors are shown for duplicate or invalid inputs.
User Login	Validates that registered users can authenticate successfully using valid credentials and are redirected to the dashboard.
Dashboard Access	Ensures authenticated users can access the main dashboard displaying expense summaries, charts, and navigation widgets.
Budget Management	Validates that users can set and save a monthly budget limit which is reflected correctly across the dashboard.
Transaction Management	Confirms that users can add new expense transactions with description, amount, and category, and that entries appear in the transaction list.
Logout	Verifies that users can log out successfully, sessions are terminated, and the browser redirects to the login page.

4.4 Test Cases

4.4.1 Registration Module Test Cases

The following test cases validate the new user registration functionality:

TC ID	Test Scenario	Precondition	Test Steps	Test Data	Expected Result	Status
TCRG-01	Valid New User Registration	Email address does not already exist in the system. Application running at http://127.0.0.1:5000.	1. Navigate to /register. 2. Click name field (id=name) and enter a valid full name. 3. Click email field (id=email) and enter a unique email. 4. Click password field (id=password) and enter a valid password. 5. Click submit button (css=.btn) to complete registration.	Name: rahul mehta Email: rahulmehta@gmail.com Password: Rahul@456	Account created. User redirected to login page with success notification.	**Pass**

TCRG -02	Registration with Existing Email	Email is already registered in the system. Application running.	1. Navigate to /register. 2. Enter a valid name in the name field. 3. Enter an already-registered email (rahulmehta@gmail.com) in the email field. 4. Enter any valid password. 5. Click submit (css=.btn). 6. Wait for the error alert element (css=.alert.alert-error) with message 'This email is already registered'.	Email: rahulmehta@gmail.com (already registered)	Error alert displayed: 'This email is already registered'.	**Pass**
TCRG -03	Registration with Invalid Input	Application is accessible.	1. Navigate to /register. 2. Enter random invalid text in name field (e.g., vwekjwek). 3. Enter improperly formatted email (no @ symbol). 4. Enter a weak/invalid password. 5. Click the submit button (css=.btn). 6. Observe validation error messages for each invalid field.	Name: vwekjwek Email: fkwefk Password: qwknfkw	Validation error messages displayed. Form submission prevented.	**Pass**

4.4.2 Login Module Test Cases

The following test cases validate the authentication functionality of the Smart Expense Tracker:

TC ID	Test Scenario	Precondition	Test Steps	Test Data	Expected Result	Status
TCLG-01	Valid User Login	User account is registered. Application running at http://127.0.0.1:5000.	1. Navigate to /login. 2. Click email field (id=identifier) and type registered email. 3. Click password field (id=password) and enter correct password. 4. Click login button (css=.btn-text). 5. Verify redirection to dashboard upon successful authentication.	Email: rahulmehta@gmail.com Password: Rahul@456	User authenticated and redirected to /dashboard.	**Pass**
TCLG-02	Invalid Login —	User account is registered. Application running.	1. Navigate to /login. 2. Click email field and	Email: rahulmehta@gmail.com	Error message displayed.	**Pass**

	Wrong Password		enter a valid registered email. 3. Click password field and enter an incorrect password. 4. Click the login button (css=.btn-text). 5. Observe that an error message is displayed and user remains on login page.	Password: 12345 (incorrect)	User remains on login page.	

4.4.3 Dashboard Module Test Cases

The following test case validates dashboard access and widget functionality:

TC ID	Test Scenario	Precondition	Test Steps	Test Data	Expected Result	Status
TCDB-01	Open Home Page Dashboard	User is authenticated and logged in.	1. Navigate to http://127.0.0.1:5000/dashboard. 2. Assert that dashboard header element (css=.dashboard-header) is present in the DOM. 3. Wait up to 5000ms for the expense summary widget (css=.summary-widget) to become visible. 4. Assert that navigation links are present. 5. Verify that charts and statistics load correctly.	Authenticated session.	Dashboard loads successfully. Summary widgets, charts, and navigation are all visible within 5 seconds.	**Pass**

4.4.4 Budget Module Test Cases

The following test case validates the budget management functionality:

TC ID	Test Scenario	Precondition	Test Steps	Test Data	Expected Result	Status
TCBG-01	Add Budget	User is authenticated and on the dashboard.	1. Navigate to http://127.0.0.1:5000/dashboard. 2. Click the 'Set Budget' button (css=.set-budget-btn). 3. Click the budget input field (id=budget-amount) and enter a valid budget value. 4. Click the confirm/save button (css=.save-budget-btn). 5. Wait up to 5000ms for the budget display widget to update. 6. Assert that the budget amount is reflected in the UI (css=.budget-display).	Budget Amount: 5000	Budget saved. Dashboard reflects the new budget limit in the budget widget.	**Pass**

4.4.5 Transaction Module Test Cases

The following test case validates the transaction recording functionality:

TC ID	Test Scenario	Precondition	Test Steps	Test Data	Expected Result	Status
TCTX-01	Add Transaction	User is authenticated and budget is set.	1. Navigate to http://127.0.0.1:5000/dashboard. 2. Click the 'Add Transaction' button (css=.add-transaction-btn). 3. Click the description field (id=txn-description) and enter a description. 4. Click the amount field (id=txn-amount) and enter the expense amount. 5. Select a category from the dropdown (id=txn-category). 6. Click submit (css=.submit-txn-btn). 7. Wait up to 5000ms and assert the new transaction appears in the transaction list (css=.transaction-list).	Description: Grocery Shopping Amount: 850 Category: Food	Transaction added. New entry visible in the transaction list. Remaining budget updates accordingly.	**Pass**

4.4.6 Logout Module Test Cases

The following test case validates session termination and logout functionality:

TC ID	Test Scenario	Precondition	Test Steps	Test Data	Expected Result	Status
TCLO-01	Logout Redirect Verification	User credentials are valid. Application running.	1. Navigate to http://127.0.0.1:5000/login. 2. Enter email (rahulmehta@gmail.com) and password (Rahul@456). 3. Click the login button (css=.btn) to authenticate. 4. After dashboard loads, click the user profile/logout button (css=.logout-btn or id=logout-link). 5. Verify that the browser redirects to the login page (/login).	Email: rahulmehta@gmail.com Password: Rahul@456	User redirected to login page. URL updated to /login.	**Pass**

4.5 Function Details

The following section describes the key application functions exercised during the testing process:

#	Function	Description
1	**Registration Function**	Facilitates creation of a new user account by collecting the user's name, email address, and password. The function validates uniqueness of the email and enforces password requirements before persisting the record to the database.
2	**Login Function**	Authenticates the user by validating the submitted email and password against stored credentials in the PostgreSQL database. Upon successful verification, the

		server initiates a user session and redirects to the dashboard. If credentials are invalid, an appropriate error message is returned.
3	**Dashboard Function**	Serves as the primary authenticated landing page, dynamically rendering the user's expense summaries, budget progress, transaction history, and analytics charts retrieved from the database.
4	**Budget Function**	Allows authenticated users to define a monthly spending budget. The value is stored in the database and compared against cumulative transactions to display the remaining budget balance on the dashboard.
5	**Transaction Function**	Enables users to record individual expense transactions by providing a description, amount, and category. Each transaction is persisted to the database and reflected in the transaction list and expense analytics.
6	**Logout Function**	Terminates the authenticated user session by clearing server-side session data. The user is redirected to the login page, and all subsequent attempts to access protected routes are denied until re-authentication.

4.6 Implementation in Selenium IDE

Installation Steps

1. Open Google Chrome and navigate to the Chrome Web Store (https://chrome.google.com/webstore).
2. Search for 'Selenium IDE' and locate the official extension published by Selenium.
3. Click 'Add to Chrome' and confirm by clicking 'Add extension' in the dialogue.
4. Once installed, the Selenium IDE icon appears in the Chrome toolbar. Click it to launch.

Recording Steps

5. Open Selenium IDE and create a new project named 'Smart Expense Tracker'.
6. Enter the base URL: http://127.0.0.1:5000.
7. Click the record icon. A new browser window opens.
8. Perform the desired user actions (navigate, enter data, click buttons).
9. Selenium IDE records each interaction as a command in the test script.
10. Click 'Stop Recording', name the test case, and save it.
11. Replay using the 'Run' button to verify all steps execute correctly.

4.7 Screenshots Section

The following figures illustrate the Selenium IDE execution logs and test results for each test case module of the Smart Expense Tracker application.

Figure 1: Valid Registration Test (TC-RG-01)

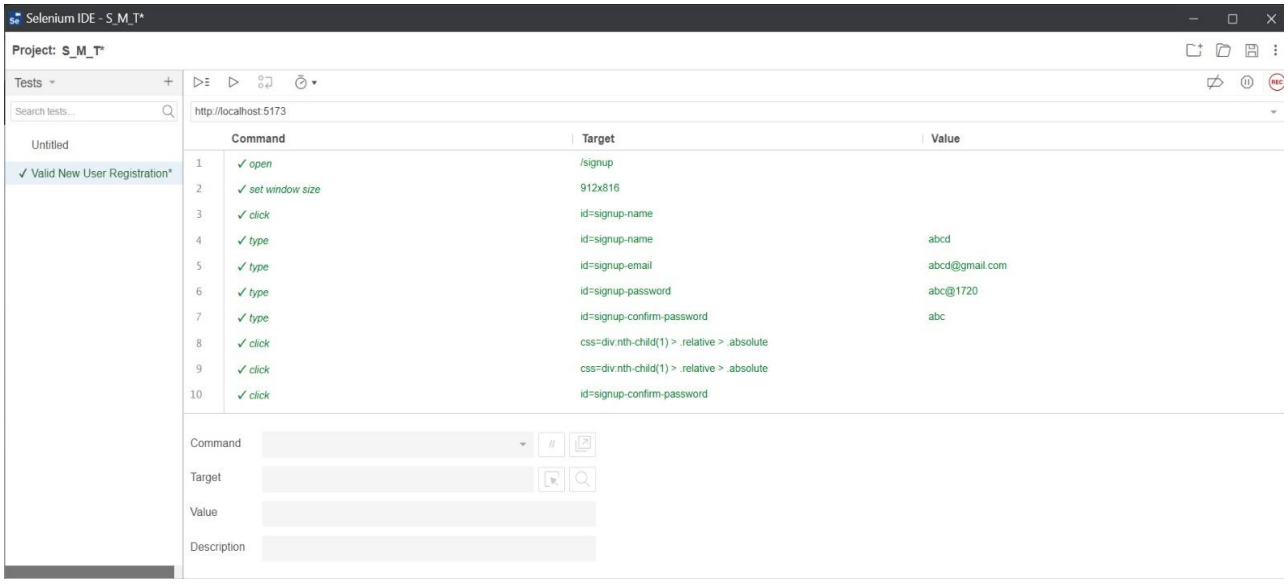


Figure 2: Existing Email Test (TC-RG-02)

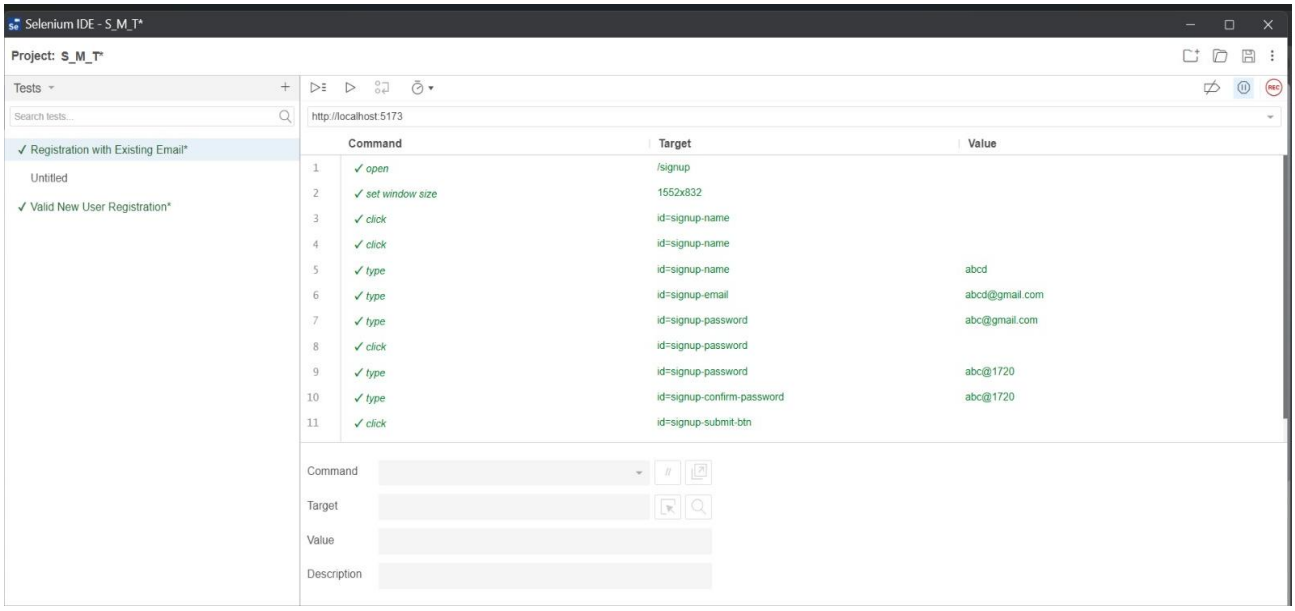


Figure 3: Invalid Input Test (TC-RG-03)

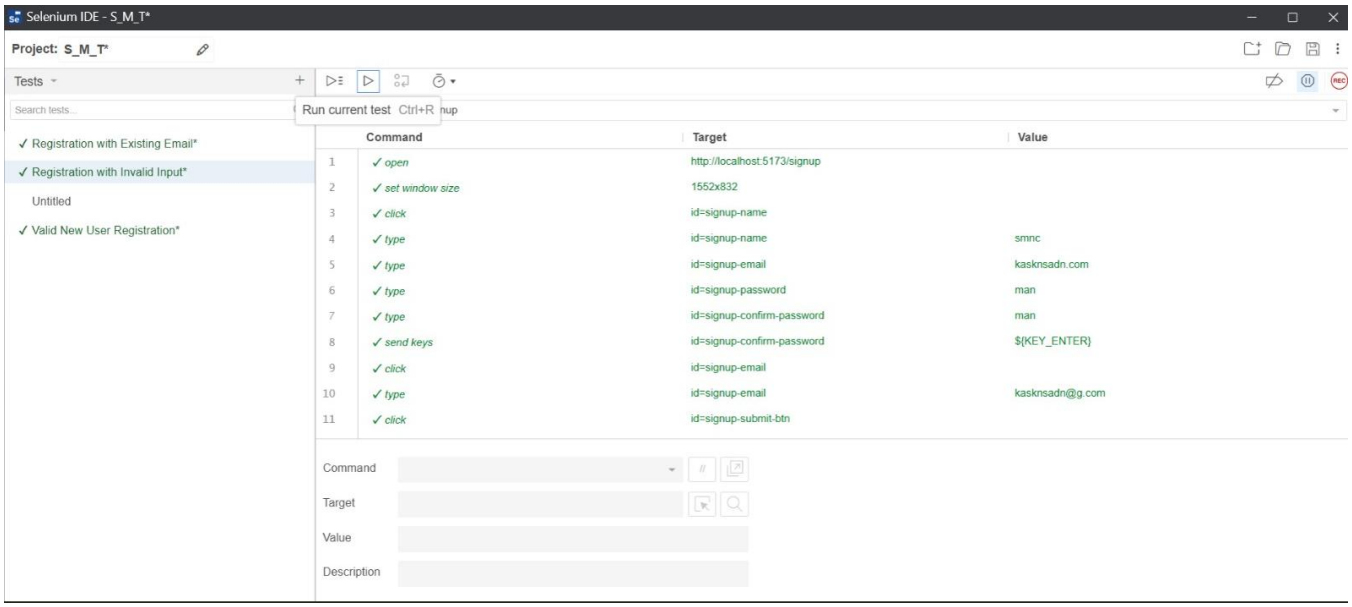


Figure 4: Valid User Login Test (TC-LG-01)

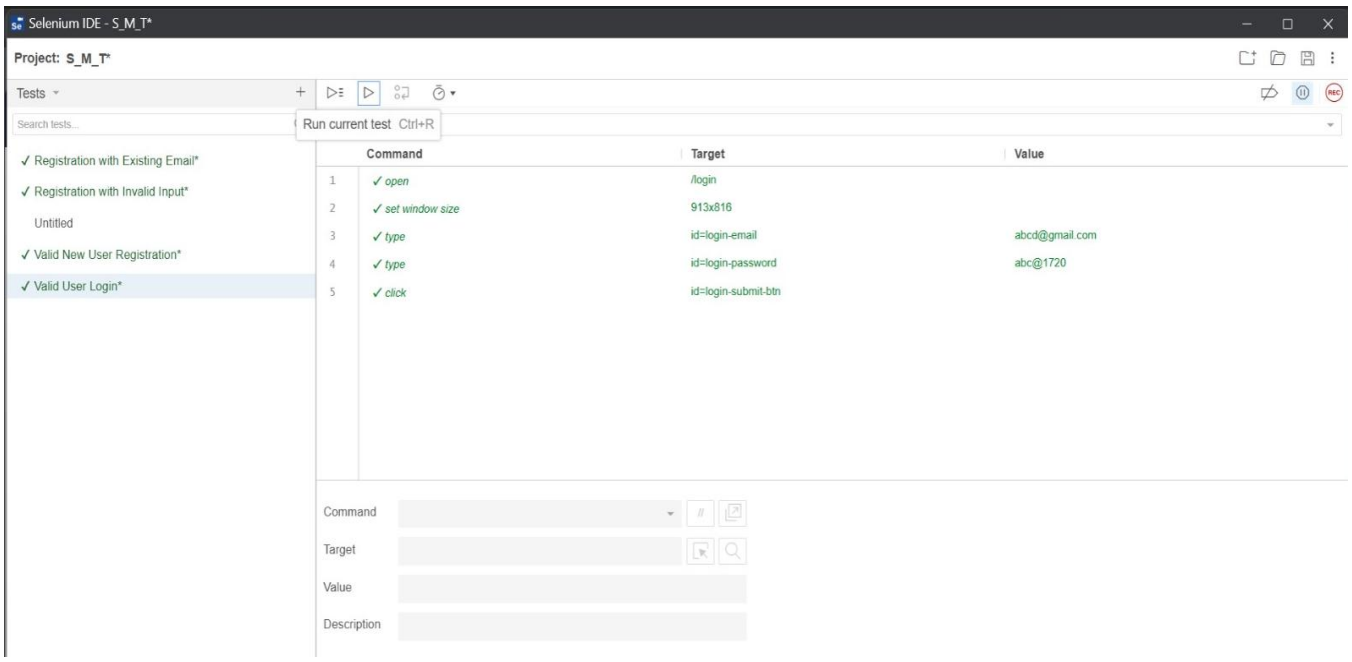


Figure 5: Invalid Login Test (TC-LG-02)

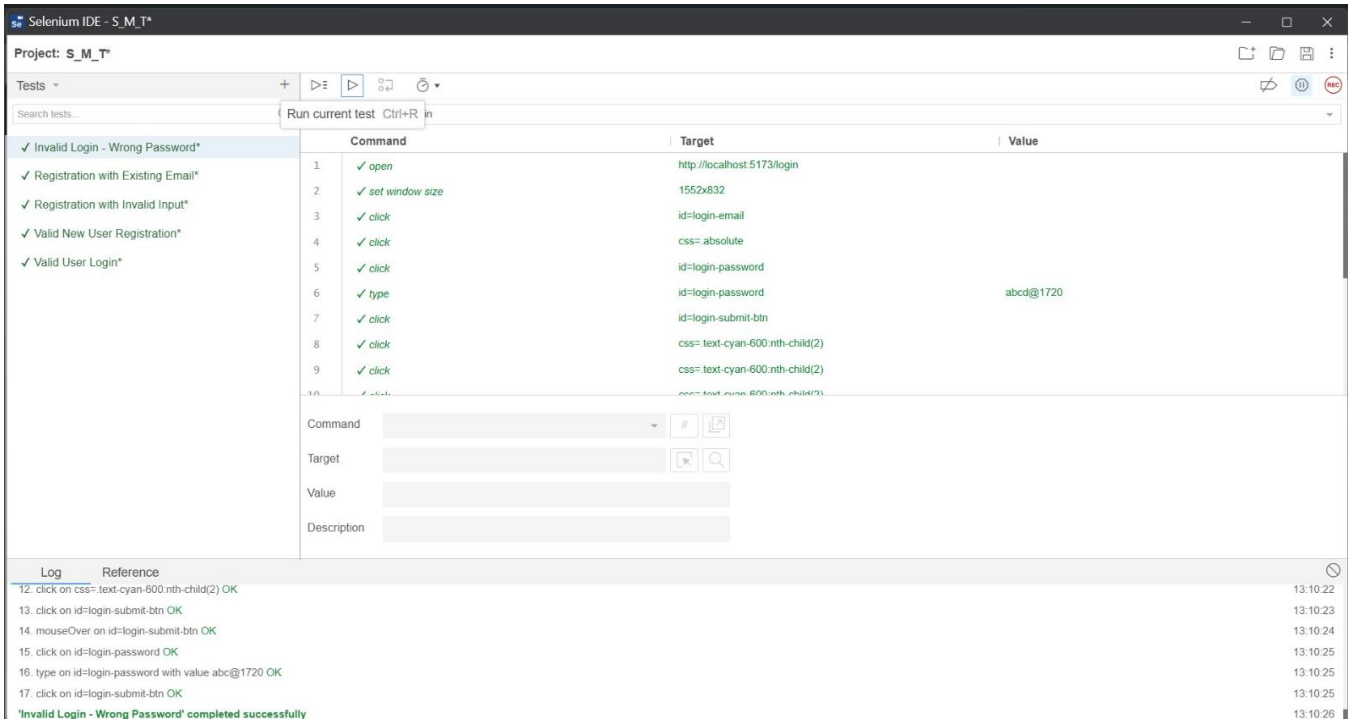


Figure 6: Open Home Page Dashboard Test (TC-DB-01)

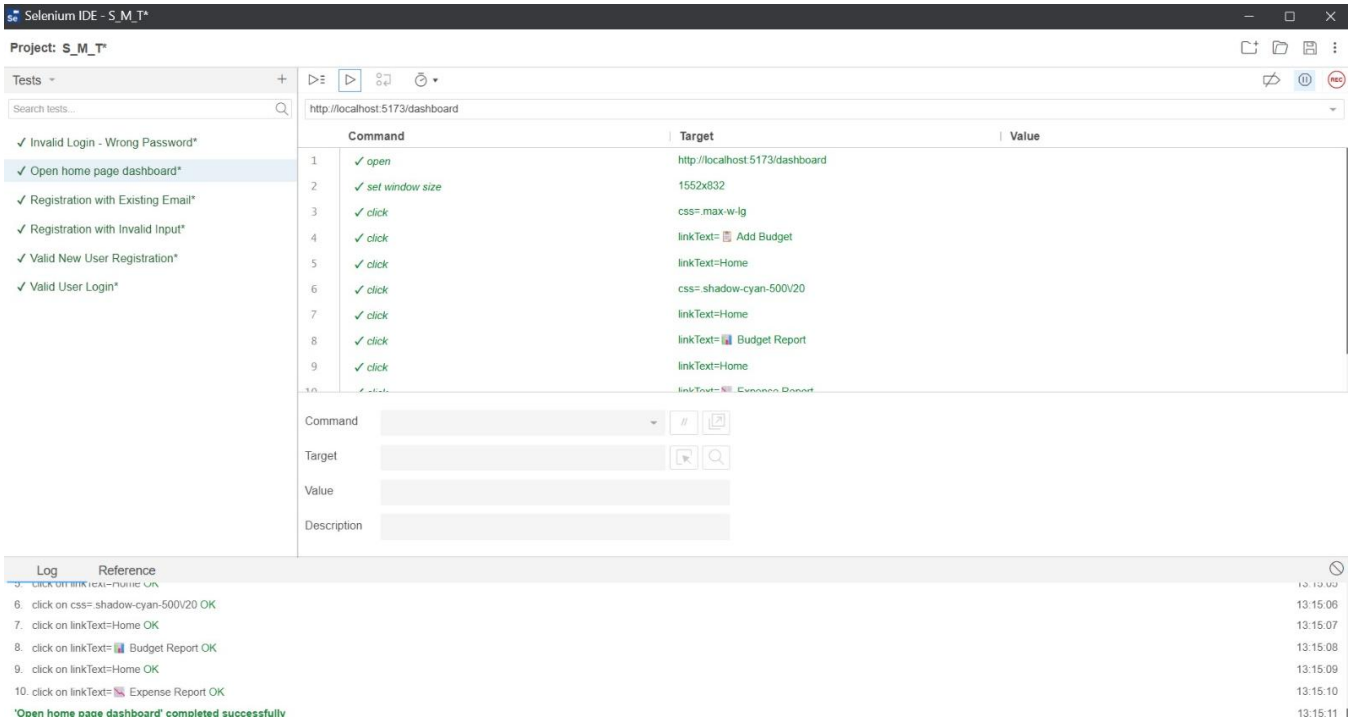


Figure 7: Add Budget Test (TC-BG-01)

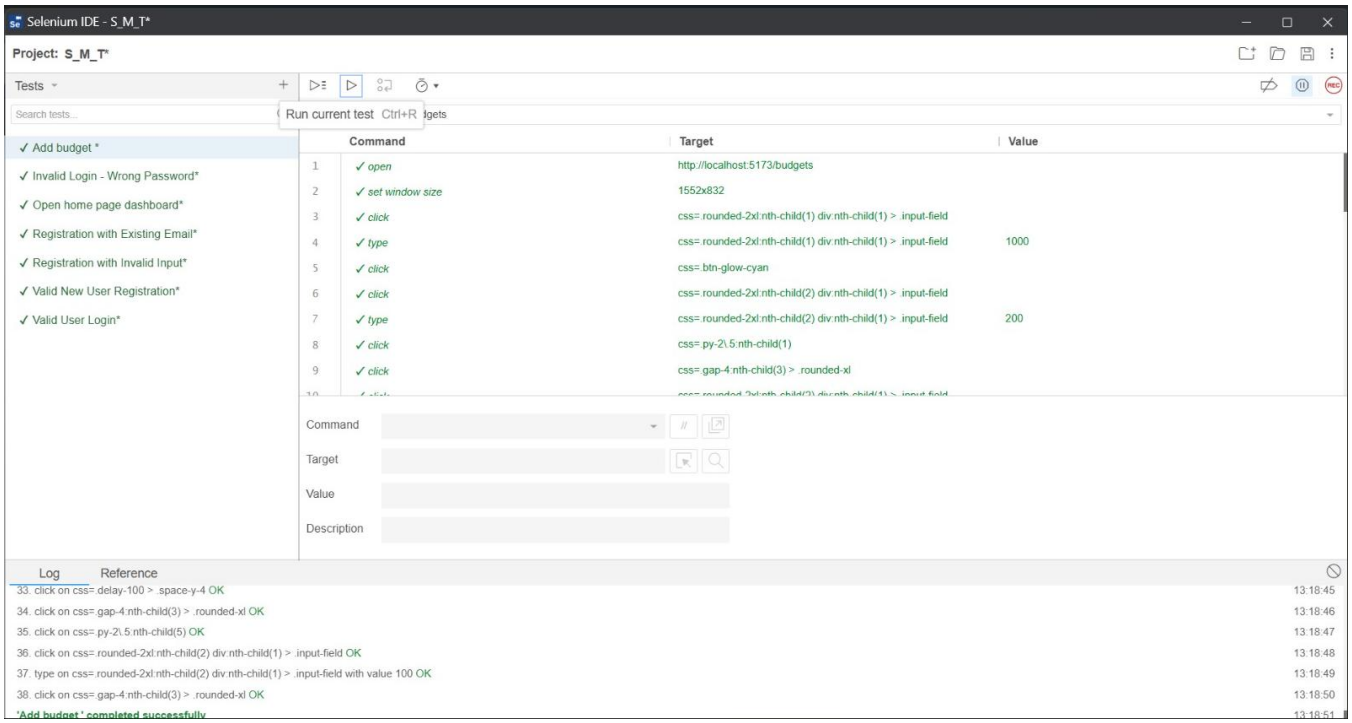


Figure 8: Add Transaction Test (TC-TX-01)

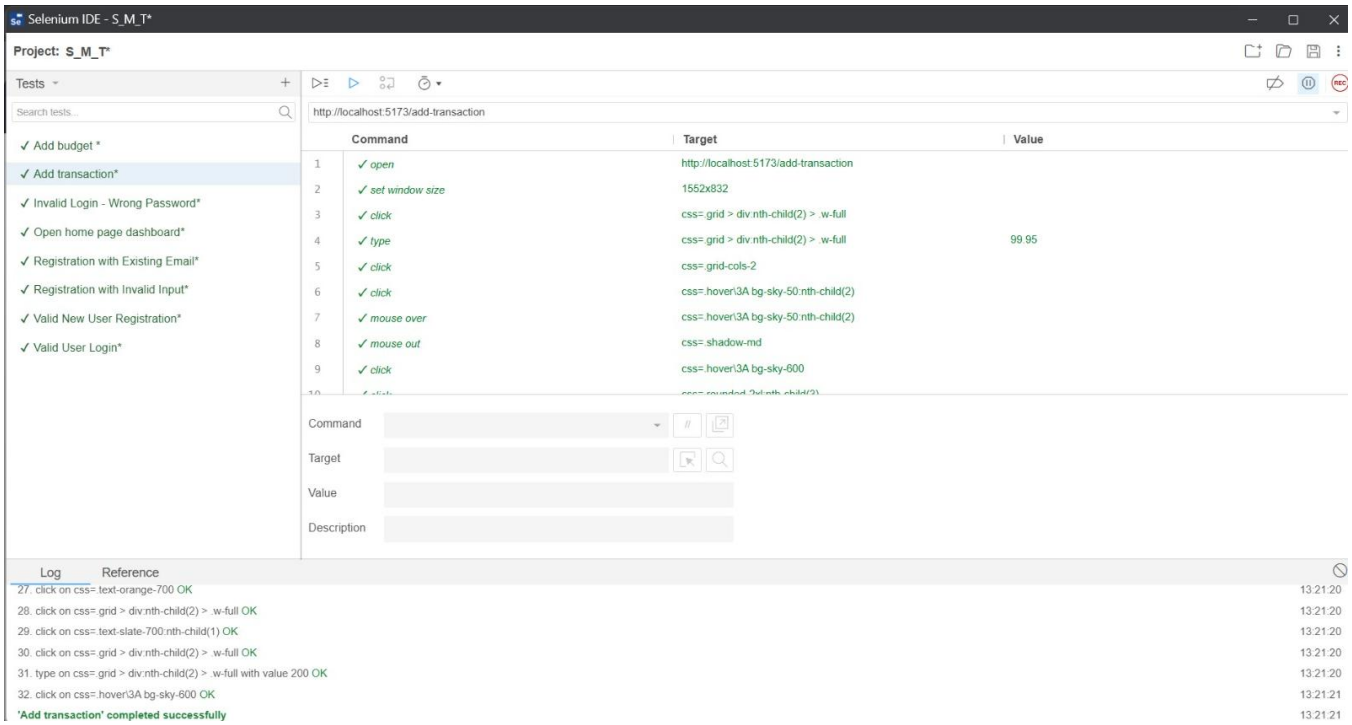
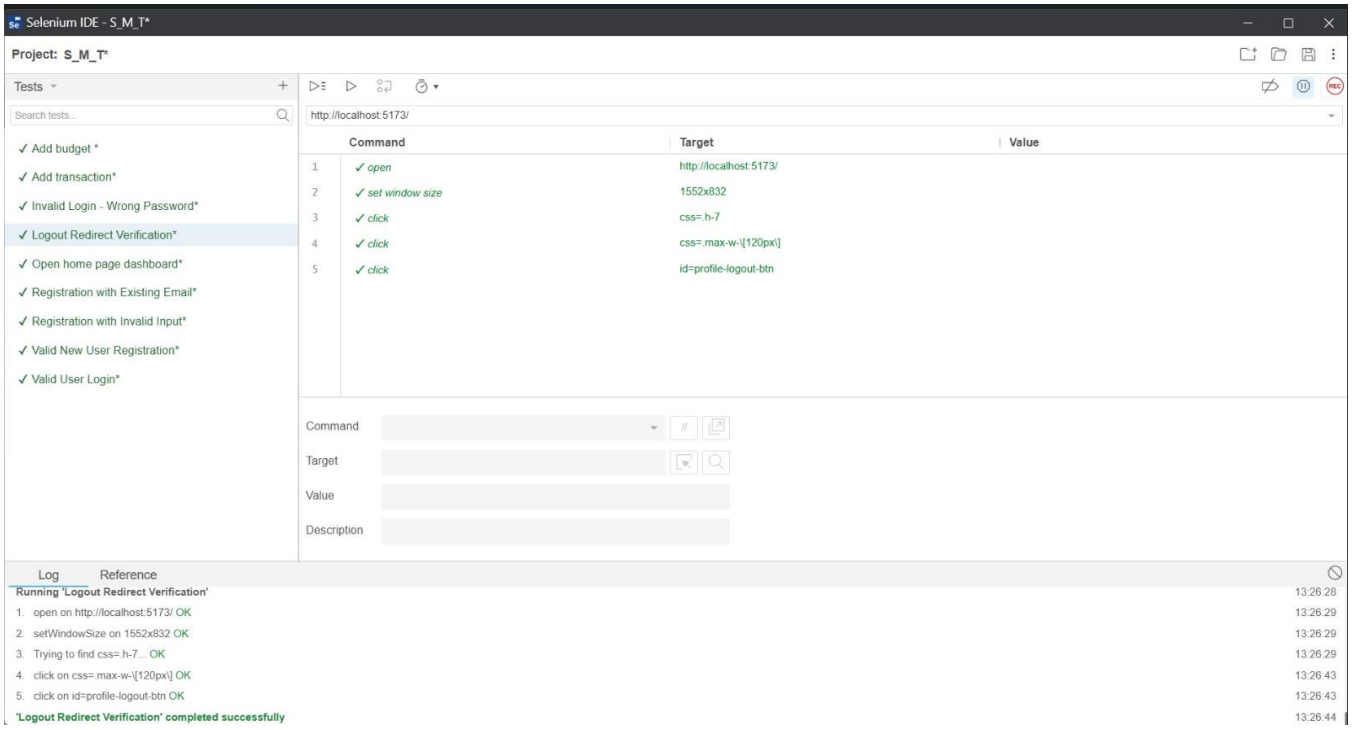


Figure 9: Logout Redirect Verification Test (TC-LO-01)



4.8 Conclusion

This laboratory experiment successfully demonstrated the application of Selenium IDE for automated testing of the Smart Expense Tracker web application. A total of nine test cases were designed and executed across six functional modules: Registration, Login, Dashboard, Budget, Transaction, and Logout. All nine test cases resulted in a Pass status, confirming that the application's core functional requirements are correctly implemented and behaving as expected.

The experiment highlighted the practical advantages of automation testing — including speed, accuracy, and repeatability — over manual testing approaches, and reinforced the importance of structured test case design with clearly defined pre-conditions, test data, and expected results. Selenium IDE proved to be an effective tool for rapid test automation without the need for programming expertise, making it well-suited for prototype and early-stage application testing.